

[Nombre de la universidad]

[Facultad / Carrera]

Sistema de biblioteca

Documento de análisis y modelado UML

Curso: Programación II

Proyecto: Proyecto Integrador — Proyecto 1

Catedrático: [Nombre del catedrático]

Semestre: 2.º semestre

Integrante	Carné
[Nombre del integrante]	[Carné]
[Nombre del integrante]	[Carné]
[Nombre del integrante]	[Carné]

Repositorio: github.com/USPG-Angels-Workspace/sistema-biblioteca

Fecha de presentación: 10 de octubre de 2026

Contenido

1. Descripción del problema: situación actual, problema identificado, propuesta de solución, objetivos, alcance y limitaciones
2. Requisitos: funcionales, no funcionales y reglas del negocio
3. Actores

4. Casos de uso: diagrama y especificaciones
5. Diagramas UML: clases, secuencia, actividad, estados y arquitectura
6. Descripción de las principales clases
7. Diseño de la persistencia en archivos JSON
8. Interfaz gráfica
9. Trazabilidad
10. Pruebas
11. Uso de GitHub y ejecución del proyecto

1. Descripción del problema

1.1 Situación actual

Una biblioteca pequeña o universitaria administra su acervo, sus lectores y sus préstamos de forma manual: libros anotados en cuadernos o en hojas de cálculo sueltas, fichas de papel para cada préstamo y cálculo de multas "de memoria" al momento de la devolución.

En este esquema:

- El personal debe revisar a mano si un libro tiene ejemplares disponibles antes de prestarlo.
- Las fechas de devolución se calculan y recuerdan manualmente, por lo que los vencimientos se detectan tarde o no se detectan.
- Las multas se cobran de forma inconsistente (distinta tarifa según quién atienda) y no queda un registro confiable de cuáles siguen pendientes.
- Obtener un dato agregado (qué libros están atrasados, cuánto se debe en multas) exige revisar todas las fichas una por una.

1.2 Problema identificado

No existe un registro único, consistente y consultable que relacione **libros**, **usuarios**, **préstamos** y **multas**. Como consecuencia:

1. Se prestan libros que ya no tienen ejemplares disponibles o a usuarios que no deberían recibirlos (con multas pendientes, con libros atrasados o por encima de su límite).
2. Los vencimientos y las multas no se controlan de forma automática ni uniforme.
3. La información se pierde o se duplica, y generar reportes es lento y propenso a errores.

1.3 Propuesta de solución

Desarrollar un **sistema de gestión de biblioteca** en C# (.NET 8) con interfaz gráfica de escritorio que:

- Registra y administra libros, usuarios (lectores y bibliotecarios), préstamos, devoluciones y multas.
- Aplica automáticamente las reglas del servicio (límites por tipo de usuario, plazos, renovaciones máximas, tarifa de multa, bloqueos por multa o atraso).
- Guarda toda la información en **archivos JSON** para que los datos permanezcan al cerrar y volver a abrir el programa.
- Genera los reportes que necesita el personal y permite exportarlos a CSV.

El sistema se diseñó siguiendo el proceso **análisis → UML → clases → código → interfaz → archivos → pruebas**; la trazabilidad de cada requisito se documenta en el capítulo 9.

1.4 Objetivo general

Desarrollar una aplicación de escritorio en C# que permita gestionar el catálogo, los usuarios, los préstamos, las devoluciones y las multas de una biblioteca, aplicando análisis orientado a objetos y modelado UML, y almacenando la información en archivos JSON.

1.5 Objetivos específicos

1. Analizar el problema y documentar la situación actual, los requisitos funcionales y no funcionales, los actores y los casos de uso.
2. Modelar la solución con diagramas UML de casos de uso, clases, secuencia y actividad, que sirvan de base para la implementación.
3. Implementar el dominio con programación orientada a objetos: clases, encapsulamiento, constructores, propiedades, métodos, herencia, polimorfismo e interfaces.
4. Implementar la persistencia en archivos JSON, garantizando que la información se recupere al reiniciar la aplicación.
5. Construir una interfaz gráfica clara con menú principal, formularios de registro, consulta, edición, eliminación, mensajes de confirmación y error, y validación de datos.
6. Verificar el funcionamiento con pruebas automatizadas y con datos de prueba que cubran todos los estados del sistema.
7. Utilizar GitHub para el versionamiento del proyecto con commits pequeños, ordenados y descriptivos.

1.6 Alcance y limitaciones

Alcance

El sistema cubre los cinco módulos definidos en el enunciado del proyecto:

Módulo	Funcionalidad incluida
Gestión de libros	Registro, edición, búsqueda, clasificación por categoría y control de disponibilidad.
Gestión de usuarios	Registro de lectores y bibliotecarios, actualización, consulta y activación/desactivación.
Gestión de préstamos	Registro de préstamos, asignación de libros, fechas de devolución y renovaciones.
Devoluciones y multas	Registro de devoluciones, control de vencimientos, cálculo y cobro de multas.
Reportes	Libros disponibles, préstamos activos, libros atrasados, usuarios y multas; exportación a CSV.

Limitaciones

- **Un solo puesto de trabajo:** los archivos JSON se cargan en memoria al iniciar; el sistema no está diseñado para que dos instancias escriban a la vez sobre los mismos archivos.
- **Sin autenticación:** el programa lo opera el personal de la biblioteca; no hay inicio de sesión ni roles con permisos diferenciados.
- **Sin reservas de libros ni notificaciones** (correo o SMS) de vencimiento.
- **Un catálogo por título:** los ejemplares de un mismo libro se manejan como una cantidad (total y disponibles), no como copias individuales con código propio.
- **Multas con una única política** (tarifa lineal por día con tope); la política es intercambiable por código (interfaz `ICalculadoraMulta`), pero no es configurable desde la interfaz.
- **Consistencia entre archivos:** cada operación escribe los archivos afectados de forma secuencial; una falla de disco entre dos escrituras podría dejar los archivos desincronizados (no se implementan transacciones).
- **Moneda:** las multas se expresan en quetzales (Q).

2. Requisitos

2.1 Requisitos funcionales

ID	Requisito	Módulo
RF-01	Registrar libros con ISBN, título, autor, editorial, año de publicación, categoría y cantidad de ejemplares.	Libros
RF-02	Editar los datos de un libro; la cantidad de ejemplares no puede ser menor que los ejemplares actualmente en préstamo.	Libros
RF-03	Buscar libros por título, autor, editorial o ISBN, y filtrarlos por categoría y por disponibilidad.	Libros
RF-04	Clasificar los libros en categorías (Ficción, No ficción, Ciencia, Tecnología, Historia, Arte, Infantil, Referencia, Otros).	Libros
RF-05	Controlar la disponibilidad: los ejemplares disponibles se descuentan al prestar y se recuperan al devolver.	Libros
RF-06	Eliminar libros que no tengan préstamos en su historial.	Libros
RF-07	No permitir dos libros con el mismo ISBN (sin importar guiones o espacios).	Libros
RF-08	Registrar lectores (con carnet) y bibliotecarios (con cargo), con nombre, DPI, correo y teléfono.	Usuarios
RF-09	Actualizar y consultar los datos de un usuario; activarlo o desactivarlo.	Usuarios

ID	Requisito	Módulo
RF-10	Validar los datos del usuario: DPI de 13 dígitos, correo con formato válido, teléfono de 8 a 15 dígitos; DPI único y carnet único.	Usuarios
RF-11	Eliminar usuarios sin préstamos ni multas en su historial; no permitir desactivar a un usuario con préstamos activos.	Usuarios
RF-12	Registrar un préstamo asignando un libro disponible a un usuario activo.	Préstamos
RF-13	Calcular automáticamente la fecha de devolución: 7 días para lectores y 14 días para bibliotecarios.	Préstamos
RF-14	Limitar los préstamos simultáneos: 3 para lectores y 5 para bibliotecarios.	Préstamos
RF-15	Impedir un préstamo si el usuario tiene multas pendientes, libros atrasados o ya tiene un ejemplar del mismo libro.	Préstamos
RF-16	Renovar un préstamo vigente hasta 2 veces; no se puede renovar un préstamo vencido o ya devuelto.	Préstamos
RF-17	Consultar los préstamos y filtrarlos por estado (Activo, Atrasado, Devuelto).	Préstamos
RF-18	Registrar la devolución de un libro y liberar el ejemplar.	Devoluciones
RF-19	Controlar los vencimientos: un préstamo pasa a "Atrasado" al día siguiente de su fecha de devolución y se muestran los días de atraso.	Devoluciones
RF-20	Calcular y generar una multa al devolver con atraso: Q2.00 por día, con un tope de Q100.00.	Multas
RF-21	Registrar el pago de una multa.	Multas
RF-22	Consultar las multas y filtrarlas por estado (Pendiente, Pagada).	Multas
RF-23	Reporte de libros disponibles.	Reportes
RF-24	Reporte de préstamos activos.	Reportes
RF-25	Reporte de libros atrasados con los días de atraso y la multa estimada.	Reportes
RF-26	Reporte de usuarios con sus préstamos activos y multas pendientes.	Reportes
RF-27	Reporte de multas con el total pendiente y el total cobrado.	Reportes
RF-28	Exportar cualquier reporte a un archivo CSV.	Reportes
RF-29	Guardar libros, usuarios, préstamos y multas en archivos JSON después de cada cambio.	Persistencia
RF-30	Recuperar toda la información desde los archivos JSON al iniciar la aplicación.	Persistencia
RF-31	Informar con un mensaje claro cuando un archivo de datos esté dañado o no se pueda leer o escribir.	Persistencia
RF-32	Ofrecer un menú principal con botones de navegación entre los módulos.	Interfaz
RF-33	Ofrecer formularios de registro y edición con validación de datos y mensajes de error.	Interfaz

ID	Requisito	Módulo
RF-34	Solicitar confirmación antes de eliminar, renovar, devolver o cobrar, y mostrar mensajes de resultado.	Interfaz

2.2 Requisitos no funcionales

ID	Requisito	Categoría
RNF-01	El sistema se desarrolla en C# sobre .NET 8.	Tecnología
RNF-02	El diseño aplica POO: encapsulamiento, constructores, propiedades, herencia (<code>Persona</code> → <code>Lector / Bibliotecario</code>), polimorfismo (límite y plazo de préstamo por tipo de usuario) e interfaces (<code>IRepositorio<T></code> , <code>IReloj</code> , <code>ICalculadoraMulta</code>).	Diseño
RNF-03	La información se almacena en archivos JSON en UTF-8, legibles y editables con cualquier editor de texto.	Persistencia
RNF-04	Cada escritura se hace en un archivo temporal y luego reemplaza al original, para no dejar archivos corruptos ante un fallo a medio guardado.	Confiabilidad
RNF-05	Las reglas del negocio viven en la capa de lógica, no en la interfaz; ningún dato inválido debe llegar a los archivos.	Integridad
RNF-06	La interfaz es clara, organizada y en español, con mensajes de error comprensibles para el usuario final.	Usabilidad
RNF-07	La aplicación se ejecuta en Windows, Linux y macOS (interfaz Avalonia).	Portabilidad
RNF-08	La lógica se puede probar sin la interfaz: el reloj es inyectable y las pruebas usan carpetas temporales.	Testabilidad
RNF-09	Las consultas y operaciones responden de forma inmediata con volúmenes de cientos de registros (datos en memoria).	Rendimiento
RNF-10	El código se versiona en GitHub con commits pequeños siguiendo Conventional Commits, con mensajes en español.	Proceso
RNF-11	La estructura del código mantiene correspondencia directa con el diagrama de clases UML.	Mantenibilidad

2.3 Reglas del negocio

ID	Regla
RN-01	Un lector puede tener hasta 3 préstamos simultáneos con plazo de 7 días.
RN-02	Un bibliotecario puede tener hasta 5 préstamos simultáneos con plazo de 14 días.
RN-03	Un préstamo se puede renovar como máximo 2 veces, y solo si no está vencido. Cada renovación agrega el plazo del tipo de usuario.

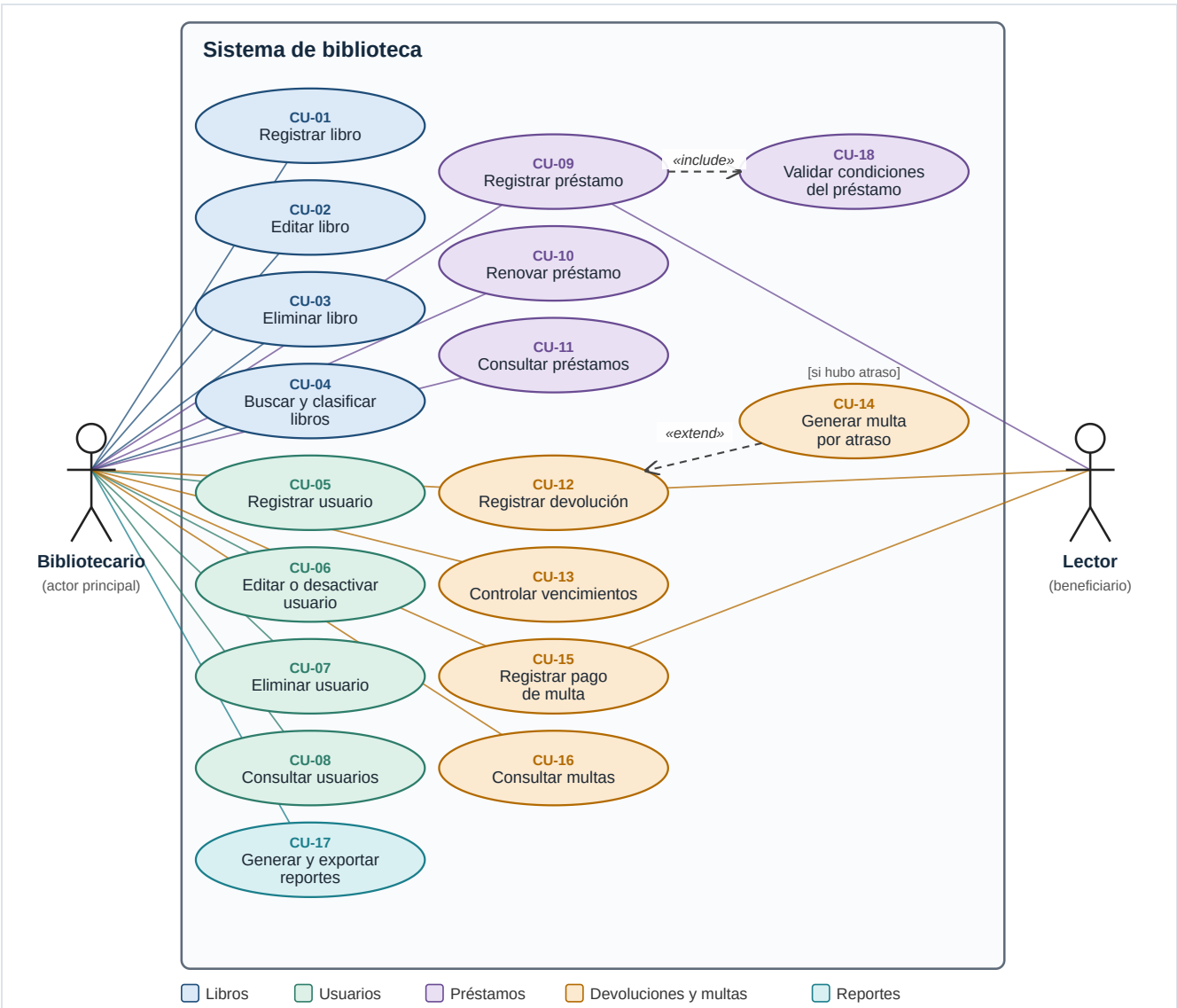
ID	Regla
RN-04	Un préstamo está Atrasado cuando la fecha actual es posterior a su fecha de vencimiento y aún no se devolvió.
RN-05	La multa es de Q2.00 por día de atraso , con un tope de Q100.00 por préstamo.
RN-06	Un usuario con multas pendientes o con préstamos atrasados no puede recibir nuevos préstamos.
RN-07	Un usuario no puede tener dos ejemplares del mismo libro en préstamo al mismo tiempo.
RN-08	Un libro o un usuario con historial de préstamos no se elimina ; un usuario se puede desactivar.
RN-09	El ISBN, el DPI y el carnet son únicos.

3. Actores

Actor	Tipo	Descripción
Bibliotecario	Principal	Persona del personal que opera el sistema: registra libros y usuarios, presta, renueva, recibe devoluciones, cobra multas y consulta reportes. Un bibliotecario también puede ser beneficiario de préstamos (con reglas propias).
Lector	Secundario	Usuario de la biblioteca que solicita libros. No opera el programa; interviene en los casos de uso de préstamo, devolución y pago de multas a través del bibliotecario.
Archivos JSON	Sistema secundario	Almacenamiento persistente (<code>libros.json</code> , <code>usuarios.json</code> , <code>prestamos.json</code> , <code>multas.json</code>) que el sistema lee al iniciar y escribe con cada cambio.

4. Casos de uso

4.1 Diagrama de casos de uso



Los casos de uso CU-09, CU-12 y CU-15 involucran al **Lector** como beneficiario del servicio, pero el sistema lo opera el **Bibliotecario**. CU-09 **incluye** siempre CU-18 (validar condiciones del préstamo); CU-14 **extiende** a CU-12 solo cuando la devolución tiene atraso. El sistema lee y escribe en los **archivos JSON** en todos los casos de uso que crean o modifican información.

4.2 Resumen de casos de uso

ID	Caso de uso	Actor principal	Requisitos
CU-01	Registrar libro	Bibliotecario	RF-01, RF-04, RF-07, RF-29

ID	Caso de uso	Actor principal	Requisitos
CU-02	Editar libro	Bibliotecario	RF-02, RF-07, RF-29
CU-03	Eliminar libro	Bibliotecario	RF-06, RF-29
CU-04	Buscar y clasificar libros	Bibliotecario	RF-03, RF-04, RF-05
CU-05	Registrar usuario	Bibliotecario	RF-08, RF-10, RF-29
CU-06	Editar o desactivar usuario	Bibliotecario	RF-09, RF-10, RF-11, RF-29
CU-07	Eliminar usuario	Bibliotecario	RF-11, RF-29
CU-08	Consultar usuarios	Bibliotecario	RF-09
CU-09	Registrar préstamo	Bibliotecario (Lector)	RF-12, RF-13, RF-14, RF-15, RF-05, RF-29
CU-10	Renovar préstamo	Bibliotecario	RF-16, RF-29
CU-11	Consultar préstamos	Bibliotecario	RF-17, RF-19
CU-12	Registrar devolución	Bibliotecario (Lector)	RF-18, RF-05, RF-29
CU-13	Controlar vencimientos	Bibliotecario	RF-19
CU-14	Generar multa por atraso (extiende CU-12)	Sistema	RF-20
CU-15	Registrar pago de multa	Bibliotecario (Lector)	RF-21, RF-29
CU-16	Consultar multas	Bibliotecario	RF-22
CU-17	Generar y exportar reportes	Bibliotecario	RF-23 a RF-28
CU-18	Validar condiciones del préstamo (incluido en CU-09)	Sistema	RF-12, RF-14, RF-15

4.3 Especificación de los casos de uso principales

CU-01 Registrar libro

Campo	Descripción
Actor	Bibliotecario
Precondición	La aplicación está abierta en el módulo <i>Libros</i> .
Flujo principal	1. El bibliotecario pulsa Nuevo libro . 2. El sistema muestra el formulario. 3. El bibliotecario ingresa ISBN, título, autor, editorial, año, categoría y ejemplares. 4. Pulsa Guardar . 5. El sistema valida los datos, verifica que el ISBN no exista y guarda el libro en <code>libros.json</code> . 6. El sistema muestra el mensaje de confirmación y actualiza la tabla.

Campo	Descripción
Flujos alternos	5a. Dato inválido o ISBN repetido: el sistema muestra el error en el formulario y no guarda. 4a. El bibliotecario pulsa Cancelar : no se guarda nada.
Postcondición	El libro queda registrado con todos sus ejemplares disponibles.

CU-09 Registrar préstamo

Campo	Descripción
Actores	Bibliotecario (opera), Lector (solicita)
Precondición	Existen al menos un usuario activo y un libro con ejemplares disponibles.
Flujo principal	1. El bibliotecario pulsa Nuevo préstamo . 2. Selecciona el usuario y el libro. 3. El sistema muestra el plazo, la fecha de devolución y los préstamos activos del usuario. 4. El bibliotecario pulsa Registrar préstamo . 5. El sistema ejecuta CU-18 (validar condiciones). 6. El sistema descuenta un ejemplar, registra el préstamo con su fecha de vencimiento y guarda <code>prestamos.json</code> y <code>libros.json</code> . 7. El sistema muestra la confirmación con la fecha de devolución.
Flujos alternos	5a. El usuario está inactivo, tiene multas pendientes, tiene libros atrasados, alcanzó su límite, ya tiene ese libro o no quedan ejemplares: el sistema muestra el motivo y no registra el préstamo.
Postcondición	Existe un préstamo en estado <i>Activo</i> y el libro tiene un ejemplar menos disponible.

CU-10 Renovar préstamo

Campo	Descripción
Actor	Bibliotecario
Precondición	Existe un préstamo en estado <i>Activo</i> (no atrasado ni devuelto).
Flujo principal	1. El bibliotecario selecciona el préstamo y pulsa Renovar préstamo . 2. El sistema pide confirmación. 3. Al confirmar, el sistema agrega al vencimiento el plazo del tipo de usuario, incrementa el contador de renovaciones y guarda <code>prestamos.json</code> . 4. Muestra la nueva fecha de devolución.
Flujos alternos	3a. Ya se usaron las 2 renovaciones permitidas: el sistema muestra el error y no cambia el préstamo.
Postcondición	El vencimiento se extendió y el préstamo sigue <i>Activo</i> .

CU-12 Registrar devolución

Campo	Descripción
Actores	Bibliotecario (opera), Lector (entrega)
Precondición	Existe un préstamo sin devolver (<i>Activo</i> o <i>Atrasado</i>).

Campo	Descripción
Flujo principal	1. El bibliotecario abre Devoluciones y multas , selecciona el préstamo y pulsa Registrar devolución . 2. El sistema muestra los días de atraso y la multa estimada, si los hay, y pide confirmación. 3. Al confirmar, el sistema registra la fecha de devolución, libera el ejemplar del libro y guarda los archivos. 4. Si hubo atraso, ejecuta CU-14. 5. Muestra el resultado de la operación.
Flujos alternos	2a. El préstamo ya había sido devuelto: el sistema informa el error.
Postcondición	El préstamo queda <i>Devuelto</i> , el libro recupera el ejemplar y, si hubo atraso, existe una multa pendiente.

CU-14 Generar multa por atraso

Campo	Descripción
Actor	Sistema (extiende a CU-12 solo cuando hay atraso)
Precondición	Se está registrando la devolución de un préstamo con fecha posterior al vencimiento.
Flujo principal	1. El sistema calcula los días de atraso (fecha de devolución – fecha de vencimiento). 2. Calcula el monto con la política de multa (Q2.00 por día, máximo Q100.00). 3. Crea la multa en estado <i>Pendiente</i> y la guarda en <code>multas.json</code> .
Postcondición	El usuario queda bloqueado para nuevos préstamos hasta pagar la multa.

CU-15 Registrar pago de multa

Campo	Descripción
Actores	Bibliotecario (opera), Lector (paga)
Precondición	Existe una multa <i>Pendiente</i> .
Flujo principal	1. En la pestaña Multas , el bibliotecario selecciona la multa y pulsa Registrar pago . 2. El sistema pide confirmación. 3. Al confirmar, marca la multa como <i>Pagada</i> con la fecha actual y guarda <code>multas.json</code> .
Postcondición	Si no quedan otras multas pendientes, el usuario puede volver a solicitar préstamos.

CU-17 Generar y exportar reportes

Campo	Descripción
Actor	Bibliotecario
Flujo principal	1. El bibliotecario abre Reportes y elige uno de los cinco reportes. 2. El sistema lo calcula con los datos actuales y lo muestra en una tabla con su resumen. 3. Opcionalmente pulsa Exportar a CSV , elige el destino y el sistema guarda el archivo.

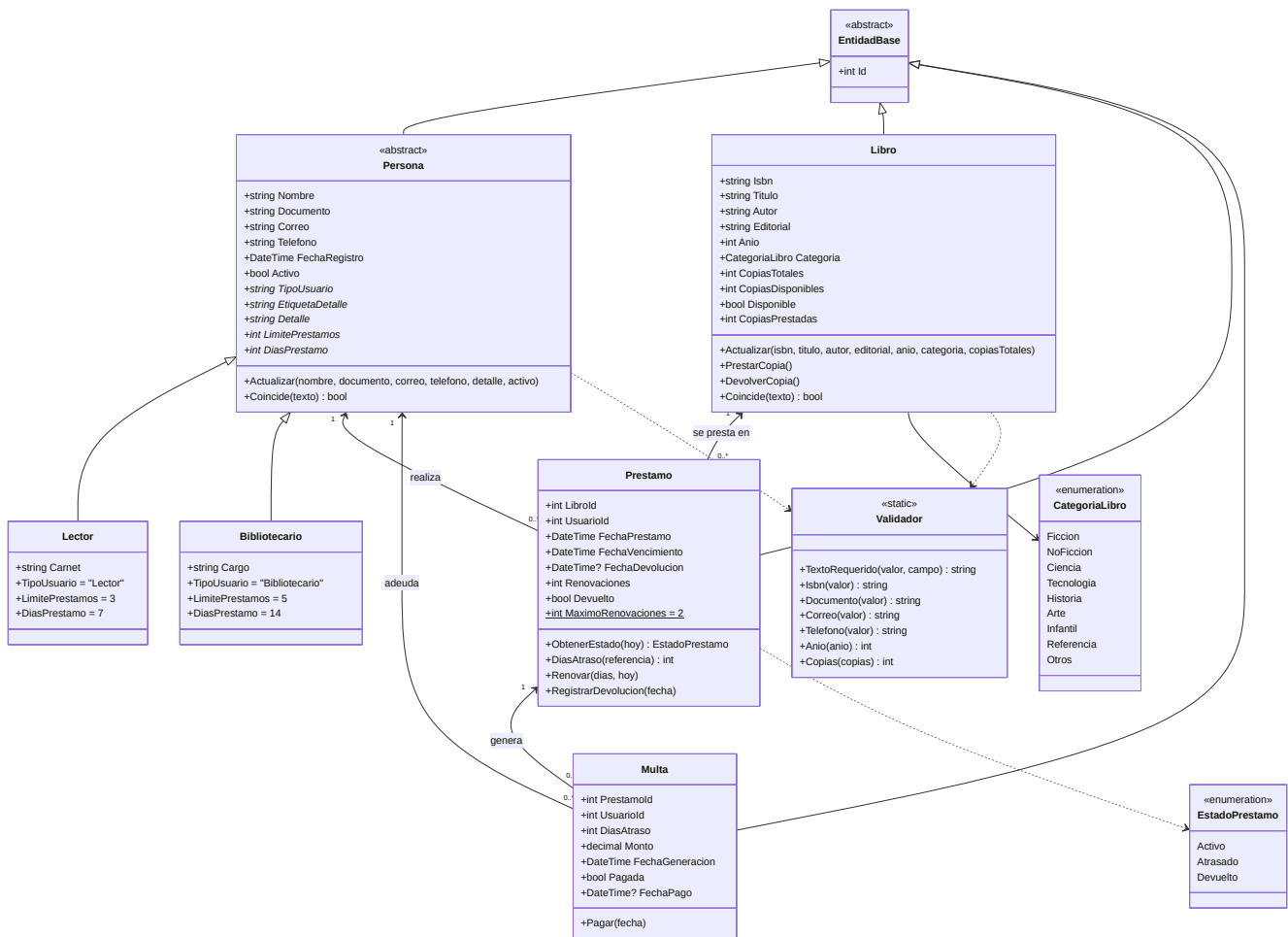
Campo	Descripción
Flujos alternos	3a. No se puede escribir en el destino: el sistema muestra un mensaje de error.
Postcondición	El reporte se visualizó y, si se pidió, se guardó como archivo CSV.

5. Diagramas UML

El diagrama de casos de uso se presentó en el capítulo 4. Este capítulo contiene los diagramas de **clases**, **secuencia**, **actividad** y **estados**, además de la vista de **arquitectura**. Los diagramas de clases, secuencia, estados y arquitectura están escritos como código Mermaid (GitHub los muestra directamente y se versionan junto con el código); los de actividad se dibujaron como imágenes SVG.

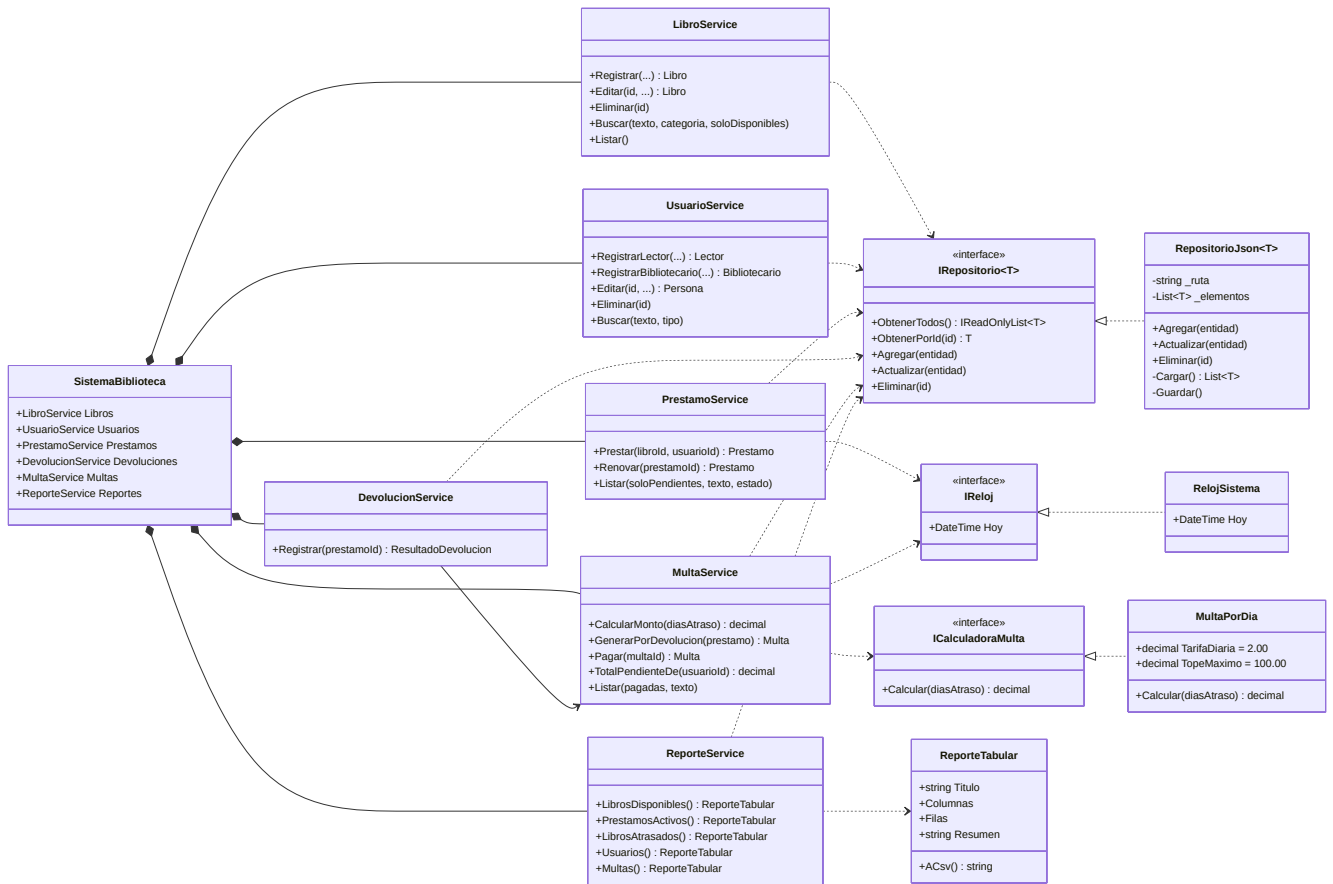
5.1 Diagrama de clases del dominio

Es el modelo que se implementó en `src/Biblioteca.Core/Modelos`. `EntidadBase` aporta el identificador que usa la persistencia; `Persona` es abstracta y sus subclases definen, por **polimorfismo**, cuántos libros puede llevar cada usuario y por cuántos días.



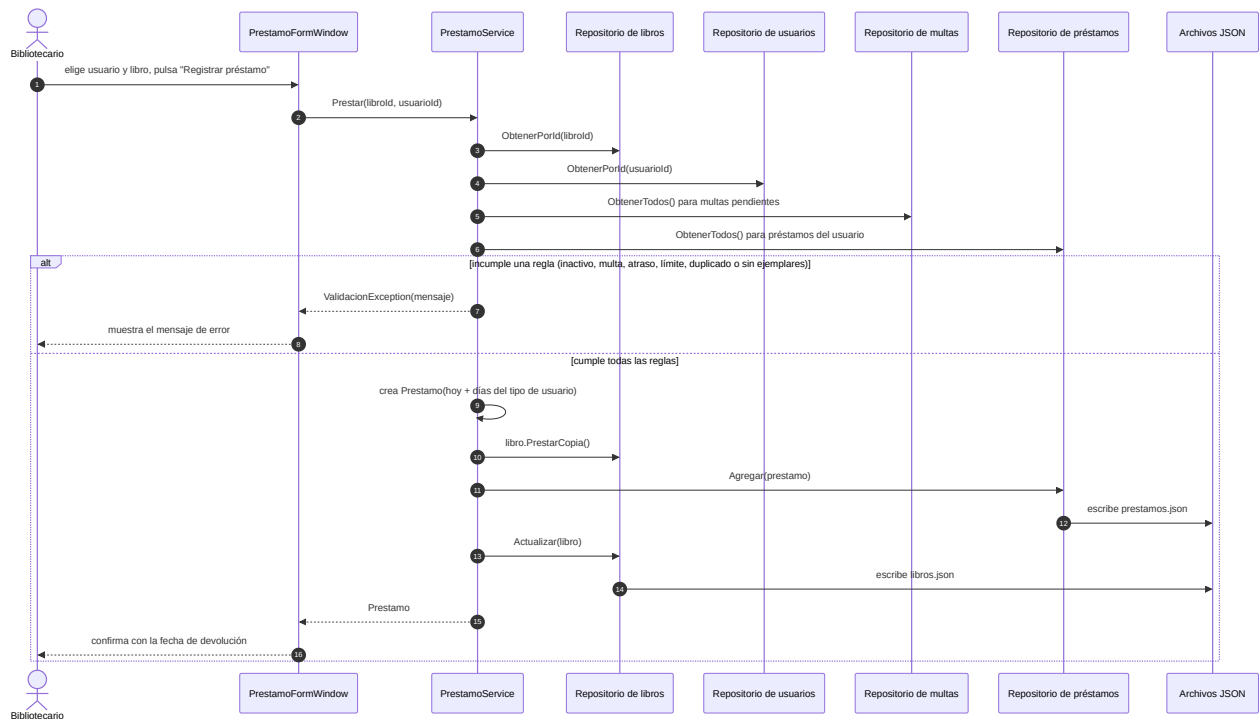
5.2 Diagrama de clases de servicios y persistencia

Muestra cómo se conectan la lógica y el almacenamiento. Los servicios dependen de **interfaces** (`IRepositorio<T>`, `IRehoy`, `ICalculadoraMulta`), lo que permite cambiar el almacenamiento, controlar la fecha en las pruebas o sustituir la política de multas sin tocar el resto del sistema.

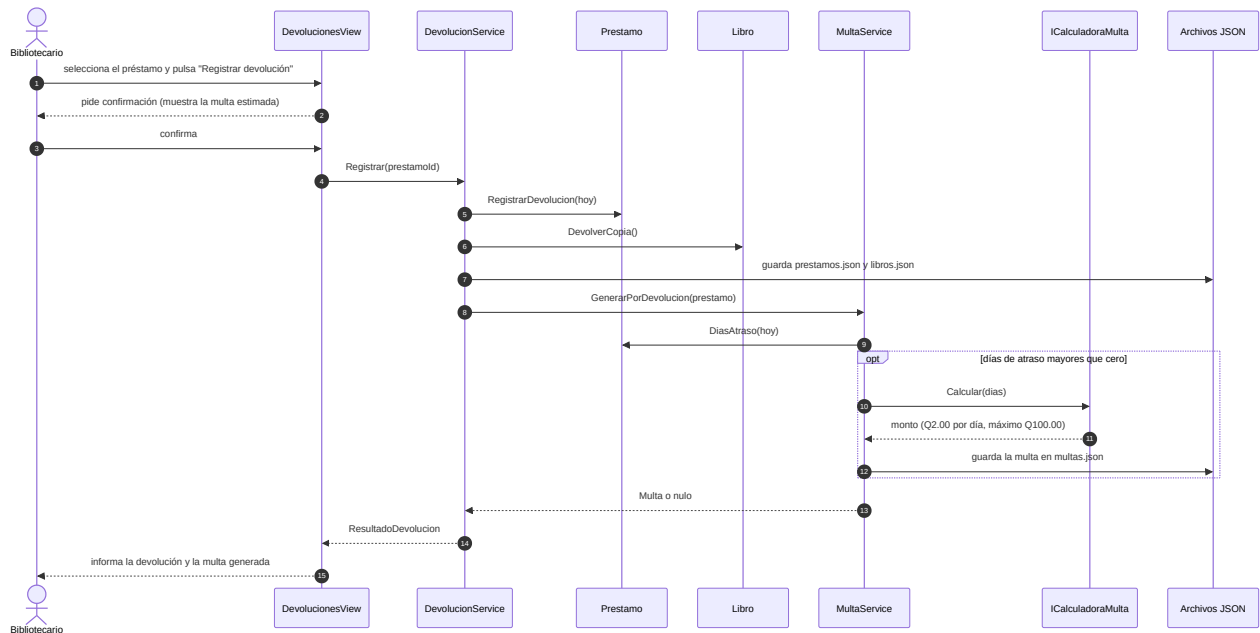


5.3 Diagramas de secuencia

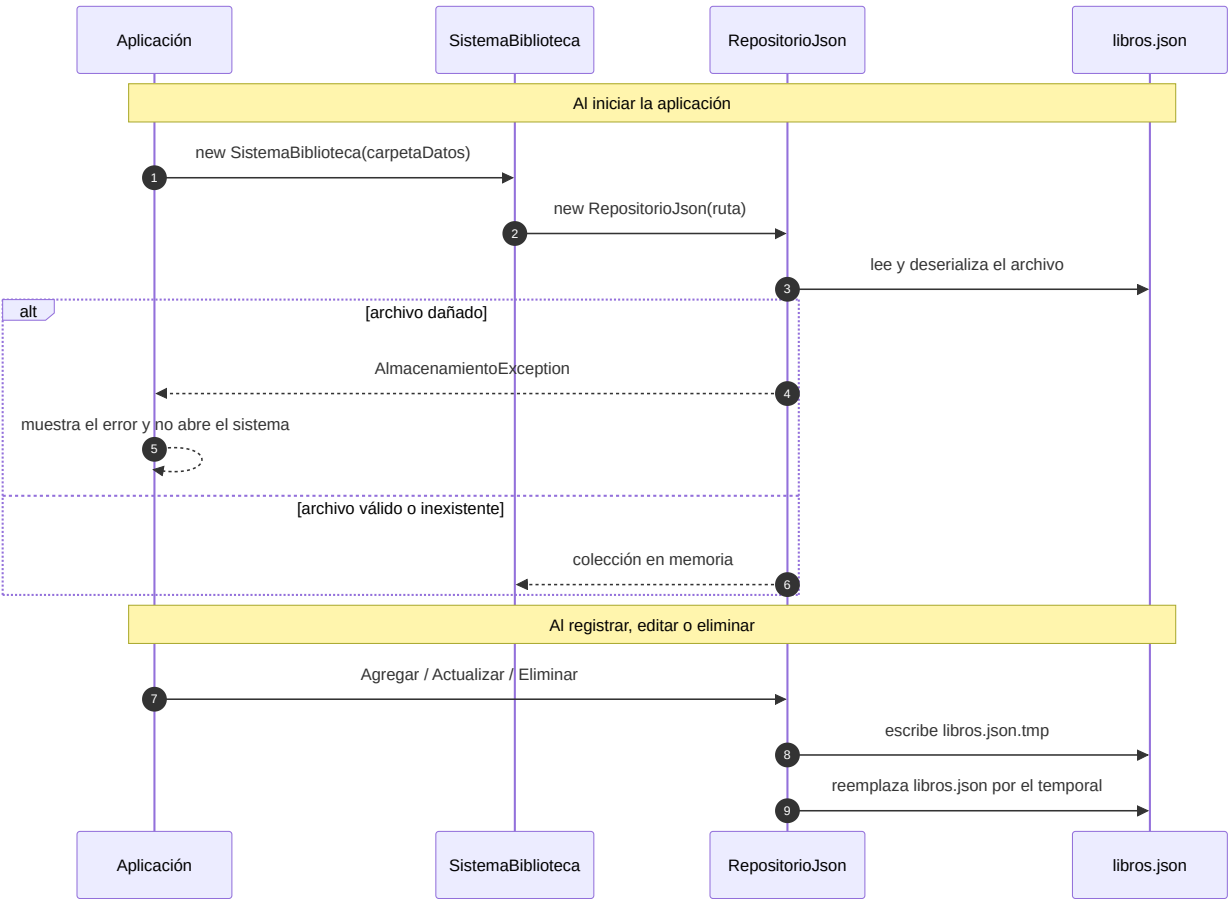
Registrar un préstamo (CU-09)



Registrar una devolución con multa (CU-12 y CU-14)



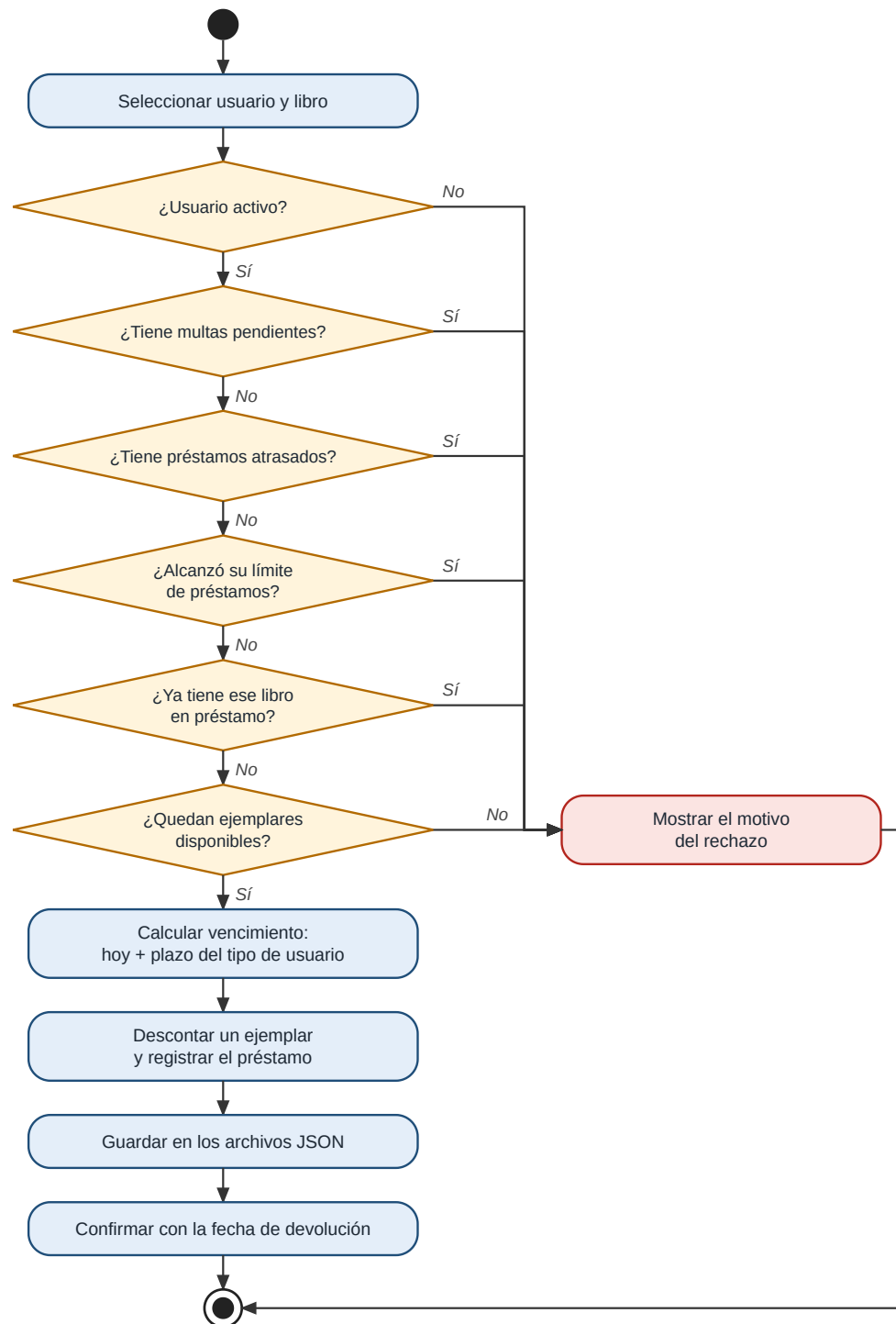
Guardar y recuperar información (persistencia)



5.4 Diagramas de actividad

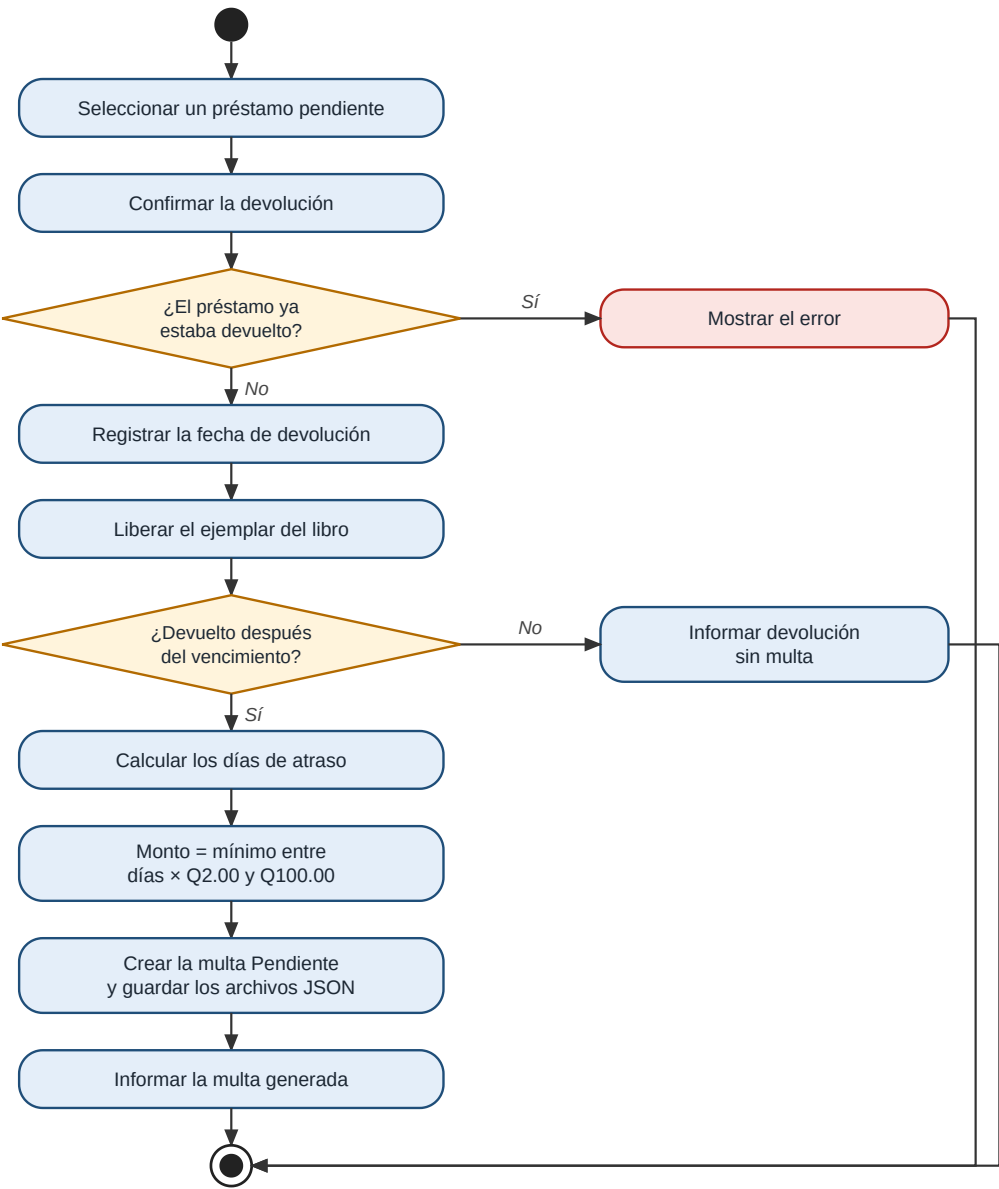
Registrar un préstamo

Diagrama de actividad: registrar un préstamo (CU-09)



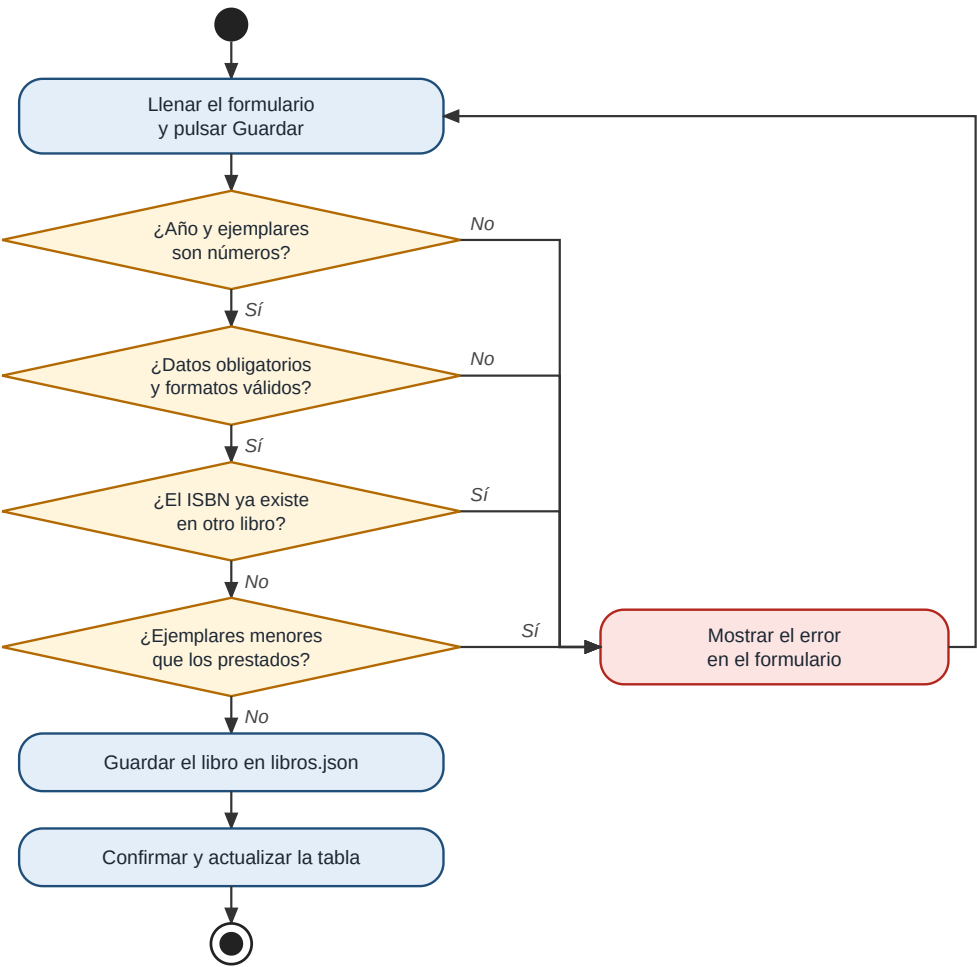
Registrar una devolución y generar la multa

Diagrama de actividad: registrar una devolución y generar la multa (CU-12 y CU-14)

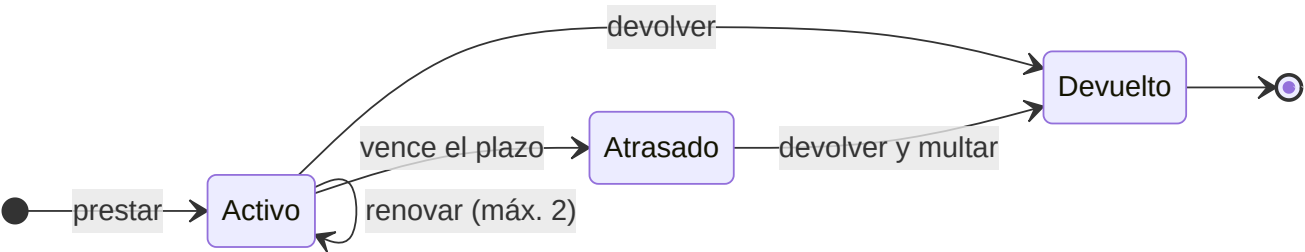


Registrar o editar un libro (validación de datos)

Diagrama de actividad: registrar o editar un libro (validación)

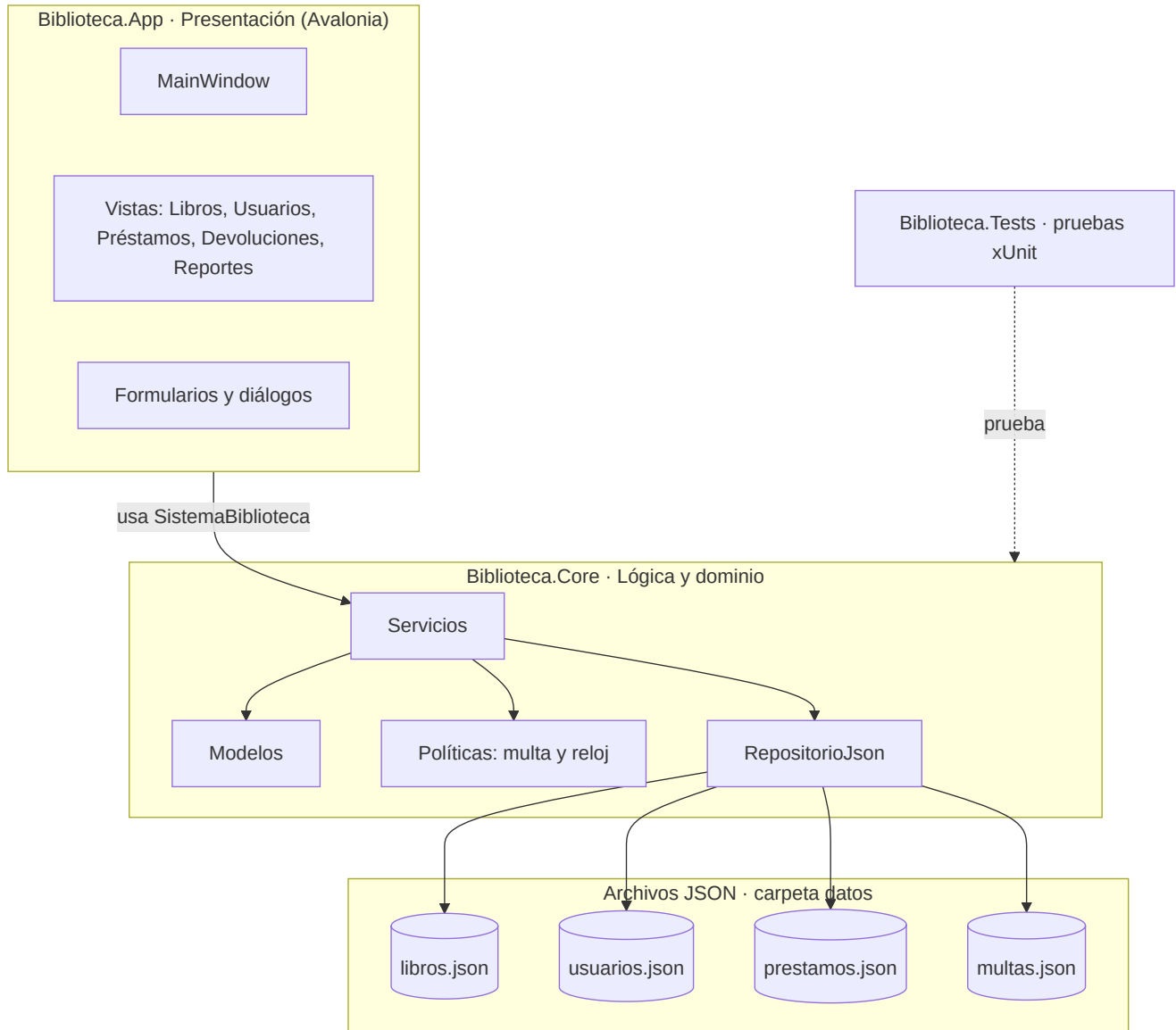


5.5 Diagrama de estados del préstamo



5.6 Arquitectura de la solución

La solución está dividida en tres proyectos con dependencias en un solo sentido (interfaz → lógica). La lógica no conoce la interfaz, por eso se puede probar de forma automática.



6. Descripción de las principales clases

Las clases del código tienen correspondencia directa con el diagrama de clases del capítulo 5 (RNF-11).

6.1 Modelos del dominio (Biblioteca.Core/Modelos)

Clase	Responsabilidad	Atributos principales	Métodos principales
EntidadBase (abstracta)	Base de todo lo que se guarda en JSON; aporta el identificador.	Id (asignado por el repositorio).	AsignarId (interno).
Persona (abstracta)	Datos comunes de quien usa la biblioteca y reglas de préstamo que cada tipo concreta.	Nombre , Documento , Correo , Telefono , FechaRegistro , Activo .	Actualizar , Coincide ; miembros abstractos TipoUsuario , LimitePrestamos , DiasPrestamo , Detalle .
Lector	Usuario que solicita libros.	Carnet .	Hereda de Persona : límite 3 préstamos, plazo 7 días.
Bibliotecario	Personal de la biblioteca (también puede pedir libros).	Cargo .	Hereda de Persona : límite 5 préstamos, plazo 14 días.
Libro	Título del catálogo con su inventario de ejemplares.	Isbn , Titulo , Autor , Editorial , Anio , Categoria , CopiasTotales , CopiasDisponibles .	Actualizar , PrestarCopia , DevolverCopia , Coincide ; calculadas Disponible , CopiasPrestadas .
Prestamo	Registro de un libro prestado a un usuario.	LibroId , UsuarioId , FechaPrestamo , FechaVencimiento , FechaDevolucion , Renovaciones .	ObtenerEstado , DiasAtraso , Renovar , RegistrarDevolucion .
Multa	Cobro generado por devolver con atraso.	PrestamoId , UsuarioId , DiasAtraso , Monto , FechaGeneracion , Pagada , FechaPago .	Pagar .
Validador (estática)	Reglas de validación compartidas (ISBN, DPI, correo, teléfono, año, ejemplares).	—	Isbn , Documento , Correo , Telefono , TextoRequerido , Anio , Copias .
CategoriaLibro , EstadoPrestamo	Enumeraciones de clasificación y de estado.	—	Texto() (nombre legible de la categoría).

6.2 Conceptos de POO aplicados

Concepto	Dónde se aplica
Clases y objetos	<code>Libro</code> , <code>Persona</code> , <code>Prestamo</code> , <code>Multa</code> y los servicios.
Encapsulamiento	Todas las propiedades tienen <code>set</code> privado; el estado solo cambia mediante métodos que validan (por ejemplo <code>Libro.PrestarCopia()</code> no permite bajar de cero ejemplares).
Constructores	Los constructores públicos validan los datos; un constructor privado sin parámetros lo usa el deserializador JSON.
Propiedades	Propiedades de solo lectura desde fuera y propiedades calculadas (<code>Libro.Disponible</code> , <code>Prestamo.Devuelto</code>).
Métodos	Comportamiento de negocio en las entidades (<code>Renovar</code> , <code>RegistrarDevolucion</code> , <code>Pagar</code>) y en los servicios.
Herencia	<code>Persona</code> → <code>Lector</code> / <code>Bibliotecario</code> ; <code>EntidadBase</code> → <code>Persona</code> , <code>Libro</code> , <code>Prestamo</code> , <code>Multa</code> .
Polimorfismo	<code>LimitePrestamos</code> y <code>DiasPrestamo</code> se resuelven según el tipo concreto de <code>Persona</code> ; <code>PrestamoService</code> los usa sin preguntar el tipo. La colección <code>List<Persona></code> se guarda y se recupera con su tipo real.
Interfaces	<code>IRepositorio<T></code> (almacenamiento intercambiable), <code>IReLoj</code> (fecha controlable en pruebas) e <code>ICalculadoraMulta</code> (política de multa sustituible).
Genéricos	<code>RepositorioJson<T></code> sirve para las cuatro colecciones.
Excepciones propias	<code>ValidacionException</code> (regla de negocio incumplida) y <code>AlmacenamientoException</code> (problema con un archivo).

6.3 Servicios (`Biblioteca.Core/Servicios`)

Servicio	Responsabilidad
<code>LibroService</code>	Registrar, editar, eliminar, listar y buscar libros; garantiza ISBN único y bloquea la eliminación de libros con historial.
<code>UsuarioService</code>	Registrar lectores y bibliotecarios, editar, desactivar, eliminar y buscar; garantiza DPI y carnet únicos.
<code>PrestamoService</code>	Registrar préstamos aplicando todas las reglas del servicio, renovarlos y listarlos con nombres y estado resueltos (<code>PrestamoDetalle</code>).
<code>DevolucionService</code>	Registrar la devolución, liberar el ejemplar y disparar la generación de la multa.
<code>MultaService</code>	Calcular y generar multas, registrar pagos, consultar multas y saldos pendientes.
<code>ReporteService</code>	Construir los cinco reportes (<code>ReporteTabular</code>) y exportarlos a CSV.

Servicio	Responsabilidad
SistemaBiblioteca	Punto de entrada: crea los repositorios JSON y los servicios; es lo único que la interfaz necesita conocer.

7. Diseño de la persistencia en archivos JSON

7.1 Archivos

Archivo	Contenido	Entidad
libros.json	Catálogo de libros con sus ejemplares.	Libro
usuarios.json	Lectores y bibliotecarios; cada objeto lleva "tipo": "lector" o "bibliotecario".	Persona
prestamos.json	Historial completo de préstamos.	Préstamo
multas.json	Multas generadas y su estado de pago.	Multa

Los archivos se guardan en la carpeta `datos/` junto al ejecutable (o en la ruta indicada por la variable de entorno `BIBLIOTECA_DATOS`). El repositorio incluye los **datos de prueba** en `datos/` y el proyecto los copia automáticamente a la carpeta de salida.

7.2 Ejemplo de formato

```
{
  "tipo": "lector",
  "carnet": "2026-0101",
  "nombre": "María Fernanda López",
  "documento": "2456789010101",
  "correo": "maria.lopez@ejemplo.com",
  "telefono": "55123401",
  "fechaRegistro": "2026-08-03T00:00:00",
  "activo": true,
  "id": 1
}
```

Solo se guardan los datos primarios: las propiedades calculadas (`LimitePrestamos`, `Disponible`, estado del préstamo, etc.) se recalculan al cargar, de modo que no puedan quedar desincronizadas.

7.3 Estrategia de lectura y escritura

1. **Al iniciar**, `RepositorioJson<T>` lee el archivo completo y lo deserializa a una lista en memoria. Si no existe o está vacío, comienza con una colección vacía.
2. **En cada cambio** (agregar, actualizar o eliminar) serializa la lista completa a `archivo.json.tmp` y luego reemplaza al archivo original. Así, un fallo a mitad de escritura no deja un archivo truncado.
3. **Identificadores**: `Id = máximo actual + 1`. Los identificadores de registros eliminados no se reutilizan mientras exista un registro con un identificador mayor.
4. **Errores**: un JSON dañado o un problema de disco se convierte en `AlmacenamientoException` con el nombre del archivo; al iniciar, la aplicación lo muestra y no abre con datos incompletos.
5. **Codificación**: UTF-8, con sangría, sin escapar tildes ni «ñ», para que el archivo sea legible.

8. Interfaz gráfica

La interfaz se construyó con Avalonia (XAML + C#), sin lógica de negocio: cada vista solo llama a los servicios y muestra el resultado. Cumple con lo mínimo solicitado:

Requisito de interfaz	Cómo se cumple
Menú principal	Barra lateral con Inicio, Libros, Usuarios, Préstamos, Devoluciones y multas, Reportes y Salir .
Botones de navegación	El menú lateral y los accesos rápidos de la pantalla de inicio.
Formularios de registro	Formularios modales de libro, usuario y préstamo.
Consulta de información	Tablas con búsqueda, filtros y ordenamiento por columna.
Edición de información	Botón Editar o doble clic sobre una fila.
Eliminación	Botón Eliminar en libros y usuarios, permitido solo sin historial.
Mensajes de confirmación	Diálogo «¿Está seguro?» antes de eliminar, renovar, devolver o cobrar; mensaje de éxito después de cada operación.
Mensajes de error	Errores de validación dentro del formulario (en rojo) y diálogo de error para operaciones rechazadas.
Validación de datos	Campos obligatorios, formatos (ISBN, DPI, correo, teléfono), números, unicidad y reglas de negocio.

Pantalla de inicio

Biblioteca

Sistema de gestión

Inicio

Libros

Usuarios

Préstamos

Devoluciones y multas

Reportes

Salir

Bienvenido

sábado, 19 de septiembre de 2026

Libros

16

28 de 36 ejemplares disponibles

Usuarios

8

8 activos

Préstamos activos

8

sin devolver

Atrasados

2

pasaron su fecha de devolución

Multas pendientes

Q14.00

por cobrar

Accesos rápidos

Registrar un préstamo

Registrar una devolución

Consultar libros

Consultar usuarios

Ver reportes

Reglas del servicio

• Lector: hasta 3 libros a la vez, 7 días de préstamo.

• Bibliotecario: hasta 5 libros, 14 días.

• Cada préstamo puede renovarse hasta 2 veces si no está vencido.

• Multa por atraso: Q2,00 por día (máximo Q100,00). Con multas pendientes o libros atrasados no se pueden hacer nuevos préstamos.

Libros

Biblioteca

Sistema de gestión

Inicio

Libros

Usuarios

Préstamos

Devoluciones y multas

Reportes

Salir

Gestión de libros

Registro, edición, búsqueda, clasificación y control de disponibilidad.

Buscar por título, autor, editorial o ISBN...

Todas las categorías

☐ Solo disponibles

ISBN	Título	Autor	Editorial	Año	Categoría	Disp.	Total	Estado
9789929000073	Breve historia del tiem	Stephen Hawk	Crítica	1988	Ciencia	1	2	Disponible
9789929000059	C# in Depth	Jon Skeet	Manning	2019	Tecnología	2	2	Disponible
9789929000141	Cálculo de una variabl	James Stewart	Cengage	2012	Ciencia	2	2	Disponible
9789929000011	Cien años de soledad	Gabriel García	Sudameric	1967	Ficción	3	3	Disponible
9789929000042	Clean Code	Robert C. Mart	Prentice Hi	2008	Tecnología	1	2	Disponible
9789929000110	Diccionario de la lengu	Real Academia	Espasa	2014	Referencia	1	2	Disponible
9789929000028	Don Quijote de la Man	Miguel de Cen	Alfaguara	1605	Ficción	2	2	Disponible
9789929000035	El principito	Antoine de Sai	Salamandr	1943	Infantil	4	4	Disponible
9789929000134	El Señor de los Anillos	J. R. R. Tolkien	Minotauro	1954	Ficción	2	3	Disponible
9789929000066	Estructuras de datos y	Mark Allen We	Pearson	2013	Tecnología	2	3	Disponible
9789929000158	Hábitos atómicos	James Clear	Paidós	2018	No ficción	1	2	Disponible
9789929000103	La historia del arte	Ernst Gombric	Phaidon	1950	Arte	1	1	Disponible
9789929000097	Popol Vuh	Anónimo	Editorial U	1550	Historia	2	2	Disponible
9789929000165	Ravuela	Julio Cortázar	Alfaguara	1963	Ficción	0	1	Sin ejemplar

Nuevo libro

Editar

Eliminar

16 libros

Formulario

Nuevo libro

ISBN (10 o 13 dígitos) *

978-9929-00-001-1

Título *

Autor *

Editorial *

Año de publicación *

Categoría *

Ficción

Ejemplares *

Cancelar

Guardar

Validación

Nuevo libro

ISBN (10 o 13 dígitos) *

123

Título *

Libro de prueba

Autor *

Autor

Editorial *

Editorial

Año de publicación *

abcd

Categoría *

Ficción

Ejemplares *

2

El año de publicación debe ser un número entero.

Cancelar

Guardar

Usuarios

Biblioteca

Sistema de gestión

Inicio

Libros

Usuarios

Préstamos

Devoluciones y multas

Reportes

Salir

Gestión de usuarios

Registro de lectores y bibliotecarios, actualización y consulta de datos.

Buscar por nombre, DPI, correo o carnet...

Todos los tipos

Tipo	Nombre	DPI	Carnet / Cargo	Correo	Teléfono	Estado
Lector	Ana Lucía Pérez	2456789030101	2026-0103	ana.perez@ejemplo.com	55123403	Activo
Lector	Carlos Alberto Méndez	2456789020101	2026-0102	carlos.mendez@ejemplo.c	55123402	Activo
Lector	José Andrés Ramírez	2456789040101	2026-0104	jose.ramirez@ejemplo.com	55123404	Activo
Lector	Luis Eduardo Morales	2456789060101	2026-0106	luis.morales@ejemplo.com	55123406	Activo
Bibliotecario	Marco Antonio Ordóñez	2456789080101	Jefe de biblioteca	marco.ordonez@ejemplo.c	55123408	Activo
Lector	María Fernanda López	2456789010101	2026-0101	maria.lopez@ejemplo.com	55123401	Activo
Bibliotecario	Rosa Elena Hernández	2456789070101	Encargada de prés	rosa.hernandez@ejemplo.	55123407	Activo
Lector	Sofía Gabriela Castillo	2456789050101	2026-0105	sofia.castillo@ejemplo.con	55123405	Activo

Nuevo usuario

Editar

Eliminar

8 usuarios

Formulario

Nuevo usuario

Tipo de usuario *

Lector

Carnet *

Nombre completo *

Documento (DPI, 13 dígitos) *

Teléfono *

Correo electrónico *

Lector: hasta 3 libros a la vez, con 7 días de préstamo.

Cancelar

Guardar

Validación

Nuevo usuario

Tipo de usuario *

Lector

Carnet *

2026-0999

Nombre completo *

Nueva Persona

Documento (DPI, 13 dígitos) *

1234

Teléfono *

55550000

Correo electrónico *

correo-malo

Lector: hasta 3 libros a la vez, con 7 días de préstamo.

El documento (DPI) debe tener exactamente 13 dígitos.

Cancelar

Guardar

Préstamos

Los préstamos atrasados se resaltan en rojo y los devueltos en gris.

Biblioteca

Sistema de gestión

Inicio

Libros

Usuarios

Préstamos

Devoluciones y multas

Reportes

Salir

Gestión de préstamos

Registro de préstamos, asignación de libros, fechas de devolución y renovaciones.

Buscar por libro o usuario...

Todos los estados

N	Libro	Usuario	Préstamo	Vence	Devuelto	Renov.	Días atraso	Estado
5	Breve historia del tie	Luis Eduardo Mor.	05/09/2026	12/09/2026		0	7	Atrasado
6	El Señor de los Anillo	Luis Eduardo Mor.	08/09/2026	15/09/2026		0	4	Atrasado
8	Estructuras de datos	Sofia Gabriela Cas	14/09/2026	21/09/2026		0	0	Activo
10	Rayuela	Carlos Alberto Mé	15/09/2026	22/09/2026		0	0	Activo
7	Clean Code	Rosa Elena Herná	10/09/2026	24/09/2026		0	0	Activo
12	Hábitos atómicos	María Fernanda L	17/09/2026	24/09/2026		0	0	Activo
9	UML y patrones	Sofia Gabriela Cas	14/09/2026	28/09/2026		1	0	Activo
11	Diccionario de la lenç	Marco Antonio Or	16/09/2026	30/09/2026		0	0	Activo
1	Cien años de soledad	María Fernanda L	24/08/2026	31/08/2026	31/08/2026	0	0	Devuelto
2	El principito	Ana Lucía Pérez	25/08/2026	01/09/2026	06/09/2026	0	5	Devuelto
3	Sapiens: de animales	José Andrés Ramír	26/08/2026	02/09/2026	09/09/2026	0	7	Devuelto
4	Clean Code	Carlos Alberto Mé	26/08/2026	02/09/2026	01/09/2026	0	0	Devuelto

Nuevo préstamo

Renovar préstamo

12 préstamos

Nuevo préstamo

Nuevo préstamo

Usuario *

Seleccione un usuario activo

Libro disponible *

Seleccione un libro

Seleccione el usuario y el libro para ver las condiciones del préstamo.

Cancelar

Registrar préstamo

Préstamo rechazado por regla de negocio

Nuevo préstamo

Usuario *

Luis Eduardo Morales · Lector (Carnet: 2026-0106)

Libro disponible *

Breve historia del tiempo · Stephen Hawking (1 disp.)

Lector: plazo de 7 días, vence el 26/09/2026.
Préstamos activos: 2 de 3 permitidos.

«Luis Eduardo Morales» tiene préstamos atrasados por devolver.

Cancelar

Registrar préstamo

Devoluciones y multas

Biblioteca

Sistema de gestión

Inicio

Libros

Usuarios

Préstamos

Devoluciones y multas

Reportes

Salir

Devoluciones y multas

Registro de devoluciones, control de vencimientos, cálculo y gestión de multas.

Devoluciones pendientes

Multas

Buscar por libro o usuario...

Nº	Libro	Usuario	Vence	Estado	Días atraso	Multa estimada
5	Breve historia del tiempo	Luis Eduardo Morales	12/09/2026	Atrasado	7	Q14.00
6	El Señor de los Anillos	Luis Eduardo Morales	15/09/2026	Atrasado	4	Q8.00
8	Estructuras de datos y análisis de	Sofía Gabriela Castillo	21/09/2026	Activo	0	—
10	Rayuela	Carlos Alberto Méndez	22/09/2026	Activo	0	—
7	Clean Code	Rosa Elena Hernández	24/09/2026	Activo	0	—
12	Hábitos atómicos	María Fernanda López	24/09/2026	Activo	0	—
9	UML y patrones	Sofía Gabriela Castillo	28/09/2026	Activo	0	—
11	Diccionario de la lengua español:	Marco Antonio Ordóñez	30/09/2026	Activo	0	—

Registrar devolución

8 pendientes de devolución, 2 atrasados

Biblioteca

Sistema de gestión

Inicio

Libros

Usuarios

Préstamos

Devoluciones y multas

Reportes

Salir

Devoluciones y multas

Registro de devoluciones, control de vencimientos, cálculo y gestión de multas.

Devoluciones pendientesMultas

Buscar por usuario o libro...

Todas las multas

N°	Usuario	Libro	Días atraso	Monto	Generada	Estado	Pagada el
2	José Andrés Ramírez	Sapiens: de animales a dios	7	Q14.00	09/09/2026	Pendiente	
1	Ana Lucía Pérez	El principito	5	Q10.00	06/09/2026	Pagada	07/09/2026

Registrar pago

2 multas — pendiente de cobro en esta lista: Q14.00

Reportes

Biblioteca

Sistema de gestión

Inicio

Libros

Usuarios

Préstamos

Devoluciones y multas

Reportes

Salir

Reportes

Libros disponibles, préstamos activos, libros atrasados, usuarios y multas.

Libros disponibles

Exportar a CSV

ISBN	Título	Autor	Categoría	Ejemplares disponibles
9789929000073	Breve historia del tiempo	Stephen Hawking	Ciencia	1 de 2
9789929000059	C# in Depth	Jon Skeet	Tecnología	2 de 2
9789929000141	Cálculo de una variable	James Stewart	Ciencia	2 de 2
9789929000011	Cien años de soledad	Gabriel García Márquez	Ficción	3 de 3
9789929000042	Clean Code	Robert C. Martin	Tecnología	1 de 2
9789929000110	Diccionario de la lengua	Real Academia Española	Referencia	1 de 2
9789929000028	Don Quijote de la Manch	Miguel de Cervantes	Ficción	2 de 2
9789929000035	El principito	Antoine de Saint-Exupéry	Infantil	4 de 4
9789929000134	El Señor de los Anillos	J. R. R. Tolkien	Ficción	2 de 3
9789929000066	Estructuras de datos y ar	Mark Allen Weiss	Tecnología	2 de 3
9789929000158	Hábitos atómicos	James Clear	No ficción	1 de 2
9789929000103	La historia del arte	Ernst Gombrich	Arte	1 de 1
9789929000097	Popol Vuh	Anónimo	Historia	2 de 2
9789929000080	Sapiens: de animales a di	Yuval Noah Harari	Historia	3 de 3

15 títulos con ejemplares disponibles (28 ejemplares).

Biblioteca
Sistema de gestión

Inicio
Libros
Usuarios
Préstamos
Devoluciones y multas
Reportes

Salir

Reportes

Libros disponibles, préstamos activos, libros atrasados, usuarios y multas.

Libros atrasados

Exportar a CSV

Libro	Usuario	Venció el	Días de atraso	Multa estimada
Breve historia del tiempo	Luis Eduardo Morales	12/09/2026	7	Q14.00
El Señor de los Anillos	Luis Eduardo Morales	15/09/2026	4	Q8.00

2 libros atrasados. Multas estimadas acumuladas: Q22.00.

Diálogos de mensajes

Confirmación	Error
 Eliminar libro ¿Eliminar el libro «Rayuela»? Esta acción no se puede deshacer. No, cancelar Sí, continuar	 No se pudo completar la operación «Luis Eduardo Morales» tiene préstamos atrasados por devolver. Aceptar

9. Trazabilidad

El proyecto demuestra la cadena **Problema → Requisitos → UML → Clases → Código C# → Interfaz → Archivos → Pruebas.**

9.1 Del problema a las clases

Problema identificado	Requisitos	Casos de uso y diagramas UML	Clases
Catálogo sin control de disponibilidad	RF-01 a RF-07	CU-01 a CU-04; diagrama de clases	Libro , LibroService , Validador
Usuarios sin registro ni límites	RF-08 a RF-11	CU-05 a CU-08; herencia y polimorfismo	Persona , Lector , Bibliotecario , UsuarioService
Préstamos a quien no debe recibirlos	RF-12 a RF-17	CU-09, CU-10, CU-11, CU-18; secuencia, actividad y estados del préstamo	Prestamo , PrestamoService
Vencimientos y multas sin control	RF-18 a RF-22	CU-12 a CU-16; secuencia y actividad de devolución	DevolucionService , MultaService , Multa , ICalculadoraMulta , MultaPorDia
Reportes lentos y propensos a error	RF-23 a RF-28	CU-17	ReporteService , ReporteTabular
Información que se pierde	RF-29 a RF-31	Secuencia de persistencia	IRepositorio<T> , RepositorioJson<T> , AlmacenamientoException
Uso poco claro del sistema	RF-32 a RF-34	Especificación de casos de uso	MainWindow , Dialogos , formularios

9.2 De las clases a la interfaz, los archivos y las pruebas

Requisitos	Código C#	Interfaz	Archivo	Pruebas
RF-01 a RF-07	LibroService.Registrar/Editar/Eliminar/Buscar , Libro.PrestarCopia/DevolverCopia	LibrosView , LibroFormWindow	libros.json	LibroTests , LibroServiceTests
RF-08 a RF-11	Persona.Actualizar , LimitePrestamos , DiasPrestamo	UsuariosView , UsuarioFormWindow	usuarios.json	PersonaTests , UsuarioServiceTests
RF-12 a RF-17	PrestamoService.Prestar/Renovar , Prestamo.Renovar/ObtenerEstado	PrestamosView , PrestamoFormWindow	prestamos.json	PrestamoTests , PrestamoFlujoTests
RF-18 a RF-22	DevolucionService.Registrar , MultaService.GenerarPorDevolucion/Pagar	DevolucionesView	multas.json , prestamos.json	MultaTests , PrestamoFlujoTests

Requisitos	Código C#	Interfaz	Archivo	Pruebas
RF-23 a RF-28	ReporteService.*, ReporteTabular.ACsv	ReportesView	(lee los cuatro archivos)	ReporteServiceTests
RF-29 a RF-31	RepositorioJson.Cargar/Guardar	Mensaje de error al iniciar	Los cuatro archivos JSON	PersistenciaTests , DatosDePruebaTests
RF-32 a RF-34	MainWindow.Navegar , Dialogos.Confirmar/Error/Informacion	Toda la aplicación	—	Verificación manual de la interfaz

10. Pruebas

10.1 Pruebas automatizadas

El proyecto `tests/Biblioteca.Tests` contiene **70 casos de prueba** (xUnit) que se ejecutan con `dotnet test` sin necesidad de abrir la interfaz:

Grupo	Qué verifica
LibroTests	Validación de ISBN, año y ejemplares; disponibilidad; edición sin bajar de los ejemplares prestados; búsqueda.
PersonaTests	Polimorfismo (límites y plazos por tipo de usuario); validación de DPI, correo y teléfono; actualización.
PrestamoTests	Fecha de vencimiento; estados; días de atraso; renovaciones y su máximo; devolución única.
MultaTests	Cálculo de la multa (tarifa diaria, tope, casos límite); pago único.
LibroServiceTests , UsuarioServiceTests	Duplicados, edición, eliminación con y sin historial, búsqueda y filtros.
PrestamoFlujoTests	Flujo completo: préstamo, límite, bloqueos por multa o atraso, renovación, devolución a tiempo y con atraso, pago de la multa.
ReporteServiceTests	Contenido de los reportes y escape de comas y comillas en el CSV.
PersistenciaTests	Los datos sobreviven al cerrar y reabrir; se conserva el polimorfismo; los identificadores continúan; no quedan temporales; archivo dañado o vacío.
DatosDePruebaTests	Los datos de prueba cargan, son consistentes (ejemplares vs. préstamos) y cubren todos los estados.

10.2 Datos de prueba

La carpeta `datos/` incluye 16 libros, 8 usuarios (6 lectores y 2 bibliotecarios), 12 préstamos y 2 multas. Se generaron con la propia lógica del sistema para que sean consistentes y cubren:

- Préstamos **activos**, **atrasados** y **devueltos** (a tiempo y con atraso).
- Un préstamo **renovado**.
- Un libro **sin ejemplares disponibles** (Rayuela).
- Una multa **pagada** y otra **pendiente** (que bloquea al usuario para nuevos préstamos).
- Un usuario con préstamos **atrasados** (Luis Eduardo Morales).
- Libros que se pueden **eliminar** (sin historial) y otros que no.

10.3 Pruebas manuales de interfaz recomendadas

1. Registrar un libro con un ISBN repetido y comprobar el mensaje de error.
2. Registrar un préstamo a un usuario con multa pendiente y comprobar el rechazo.
3. Devolver un préstamo atrasado y comprobar que se genera la multa correcta.
4. Pagar la multa y volver a prestar al mismo usuario.
5. Cerrar la aplicación, abrirla de nuevo y comprobar que los datos siguen ahí.
6. Exportar un reporte a CSV y abrirlo en una hoja de cálculo.

11. Uso de GitHub y ejecución del proyecto

11.1 Repositorio

Repositorio: <https://github.com/USPG-Angels-Workspace/sistema-biblioteca>

```
sistema-biblioteca/  
├─ Biblioteca.sln  
├─ datos/           archivos JSON con los datos de prueba  
├─ docs/           documento de análisis, diagramas, imágenes y presentación  
├─ src/  
│   ├─ Biblioteca.Core/  dominio, servicios y persistencia  
│   └─ Biblioteca.App/   interfaz gráfica (Avalonia)  
└─ tests/  
    └─ Biblioteca.Tests/  pruebas unitarias (xUnit)
```

11.2 Flujo de trabajo

- Los commits siguen **Conventional Commits** con la descripción en español (`feat` , `fix` , `docs` , `chore` , `test` ...).

- Cada commit contiene un solo cambio lógico y el historial se construyó por capas: estructura base → dominio → persistencia → servicios → pruebas → interfaz → documentación.

11.3 Cómo ejecutar

Requisitos: .NET SDK 8 o superior.

```
dotnet build Biblioteca.sln  
dotnet test tests/Biblioteca.Tests  
dotnet run --project src/Biblioteca.App
```